Exercise Sheet 11

Before the Throne

House rules.

- First, inside the lesson folder: `pip install -r requirements.txt` (pandas, openpyxl, matplotlib).
- One file per exercise in `my_work/`. Everything you save goes to `my_work/output/`; create that folder at the top of every script with `Path("my_work/output").mkdir(parents=True, exist_ok=True)`.
- Every chart script starts with `import matplotlib` and `matplotlib.use("Agg")` **before** `import matplotlib.pyplot as plt`. Save with `plt.savefig(...)`; never `plt.show()`.
- Compare the printed output with the sheet, then **open the PNG or the workbook** and look at it. The file is the real result.
- Hand in: commit and push to `lesson-11`, open a pull request, and attach your favourite chart to it. The Privy Council likes pictures.

Exercise 1 Your first chart

10 min · `ex1_first_chart.py`

> **RAVEN #1101 from Queen Aldara, at dusk:** *"The Privy Council cannot read tables. We desire a picture of how many subjects serve in each guild. A bar thing. Blue, if possible. A."*

GIVEN

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
df = pd.read_csv("data/thornmere_payroll.csv")
```

(a) Count members per guild with `value_counts()`, biggest first, and print the result.

(b) Draw a bar chart of it (`plt.bar(counts.index, counts.values)`), with a title and a y-axis label. Rotate the guild names 30° so they don't overlap: `plt.xticks(rotation=30, ha="right")`.

(c) Call `plt.tight_layout()`, then save as `my_work/output/headcount.png` at `dpi=150`. Open it.

EXPECTED OUTPUT

```
department
Forge         60
Guard         35
Harbour       25
Clockworks    20
Treasury      18
Library       15
Apothecary    15
Kitchens      12
Name: count, dtype: int64
Saved my_work/output/headcount.png
```

Hint: The Queen's blue is `color="#1f3a5f"`. Any colour name works too: `"steelblue"`, `"orange"`.

Exercise 2 — Where do salaries sit?

10 min · `ex2_salary_histogram.py`

> **RAVEN #1102 from Lady Siv of the Rewards:** *"Before the pay review I want to see the salaries, not scroll through them. How many subjects earn roughly what? And where's the middle?"*

Same Given block as Exercise 1.

(a) Print the lowest, highest and median salary, formatted as crowns (`cr`) with thousands separators.

(b) Draw a histogram of `annual_salary` with `bins=12`. Add a dashed vertical line at the median: `plt.axvline(median, linestyle="--", label="Median")` and `plt.legend()`.

(c) Label both axes, add a title, save as `my_work/output/salaries.png`.

EXPECTED OUTPUT

```
Lowest salary:  10,300 cr
Highest salary: 68,750 cr
Median salary:  29,375 cr
Saved my_work/output/salaries.png
```

Hint: A histogram is a bar chart of *how many* values fall into each bucket. Excel can do one, but only since 2016 and only if you find the button.

Exercise 3 — Side by side

15 min · `ex3_side_by_side.py`

> **RAVEN #1103 from Mistress Pell:** *"Average salary per guild, men next to women, one picture. The Privy Council sits in two weeks and I'd like them to notice something. P."*

Same Given block, plus `import numpy as np`.

(a) Build a pivot table: guilds as rows, gender as columns, mean `annual_salary`, rounded to 0 decimals. Print it.

(b) Grouped bars: `x = np.arange(len(table.index))`, `width = 0.4`. Draw women at `x - width/2` and men at `x + width/2`, each with a `label=`. Put the guild names on the ticks with `plt.xticks(x, table.index, rotation=30, ha="right")`.

(c) Y-axis label, title, legend, `figsize=(9, 4.5)`, save as `my_work/output/salary_by_guild_gender.png`.

(d) Look at the picture. Which guild would you ask Mistress Pell about first?

EXPECTED OUTPUT

```
gender            F        M
department
Apothecary  33594.0  32357.0
Clockworks  31264.0  45342.0
Forge       26372.0  30536.0
Guard       27150.0  35674.0
Harbour     19075.0  34561.0
Kitchens    27631.0  44125.0
Library     29555.0  34638.0
Treasury    28267.0  36828.0
Saved my_work/output/salary_by_guild_gender.png
```

Exercise 4 Back to Excel

10 min · `ex4_first_workbook.py`

> **RAVEN #1104 from Orrin of the Clockworks:** *"The Queen's chamberlain asked if the Python thing 'does Excel'. I said yes. Please make me right."*

GIVEN

```
import pandas as pd
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
df = pd.read_csv("data/thornmere_payroll.csv")
df["is_woman"] = df["gender"] == "F"
```

(a) Group by guild (the `department` column) and build a table with `members` (count of `employee_id`), `women` (sum of `is_woman`) and `avg_salary` (mean, rounded to 0). Sort by members, biggest first.

(b) Add a column `pct_women` = women / members, rounded to 3 decimals. Print the table.

(c) Save it with `to_excel("my_work/output/headcount.xlsx", sheet_name="Headcount")`. Open the workbook.

EXPECTED OUTPUT

```
             members  women  avg_salary  pct_women
department
Forge             60     20     29148.0      0.333
Guard             35     14     32264.0      0.400
Harbour           25      6     30844.0      0.240
Clockworks        20      7     40415.0      0.350
Treasury          18      9     32547.0      0.500
Apothecary        15      8     33017.0      0.533
Library           15     11     30910.0      0.733
Kitchens          12      8     33129.0      0.667
Saved my_work/output/headcount.xlsx
```

Hint: `True` counts as 1 when you sum it. That's why `is_woman` is handy.

Exercise 5 One workbook, several sheets

10 min · `ex5_several_sheets.py`

> **RAVEN #1105 from Mistress Pell:** *"One file, please, not six. Tabs are fine. Tabs are great. Dalvric once sent me fourteen workbooks called final.xlsx. By raven. Separately."*

Same Given block as Exercise 4 (without `is_woman`).

(a) Make three tables: salary `count`, `mean`, `median` by guild; the same by location (both rounded to 0); and the top 10 earners with `employee_id, first_name, last_name, department, annual_salary`.

(b) Write all three into `my_work/output/salary_report.xlsx` using `with pd.ExcelWriter(path) as writer:` and one `to_excel(writer, sheet_name=...)` per table. The top 10 sheet gets `index=False`.

(c) Print the location table and a line naming the sheets.

EXPECTED OUTPUT

```
                count     mean   median
location
Harbourtown        40  30158.0  28250.0
Thornmere Keep     93  31834.0  30000.0
Westmarch          67  33328.0  29950.0
Sheets written: By guild, By location, Top 10
```

Exercise 6 Make it pretty

15 min · `ex6_make_it_pretty.py`

> **RAVEN #1106 from Queen Aldara:** *"Lovely numbers. Could the top row be bold? And blue? And could the crowns look like crowns? And could the columns not be squashed? We thank you ever so. A."*

GIVEN

```
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
df = pd.read_csv("data/thornmere_payroll.csv")
```

(a) Per guild: `members`, `avg_salary` (mean), `total_bonus` (sum), sorted by average salary, highest first. Add `bonus_per_head` = total bonus / members. Save to `my_work/output/pretty_report.xlsx`, sheet `Guilds`.

(b) Re-open it with `load_workbook(path)`, pick the sheet with `wb["Guilds"]`, and for every cell in row 1 set `Font(bold=True, color="FFFFFF")` and `PatternFill("solid", fgColor="1F3A5F")`.

(c) Give columns C to E the number format `'#,##0.00 "cr"'` (single quotes outside, because the format itself contains double quotes): loop over `ws.iter_rows(min_row=2, min_col=3, max_col=5)`.

(d) Set column A to width 16 and B–E to 15 (`ws.column_dimensions["A"].width`), freeze the header with `ws.freeze_panes = "A2"`, save, and print the checks below.

EXPECTED OUTPUT

```
Rows: 9, columns: 5
A1 bold: True, C2 format: #,##0.00 "cr"
Saved my_work/output/pretty_report.xlsx
```

Hint: `ws.max_row`, `ws.max_column`, `ws["A1"].font.bold` and `ws["C2"].number_format` give you the four values to print.

Exercise 7 A chart inside the workbook STRETCH

15 min · `ex7_chart_in_excel.py`

> **RAVEN #1107 from Queen Aldara:** *"Can the picture live in the Excel? The Privy Council likes it when things are in the Excel. A."*

(a) Average salary per guild, rounded, sorted smallest to largest. Print it. Draw a **horizontal** bar chart (`plt.barh`) and save it as `my_work/output/avg_salary.png`.

(b) Write the table to `my_work/output/report_with_chart.xlsx`, sheet `Data` (`avg.to_frame("avg_salary").to_excel(...)`).

(c) Re-open the workbook, `ws = wb.create_sheet("Chart")`, then `ws.add_image(Image("my_work/output/avg_salary.png"), "B2")` with `from openpyxl.drawing.image import Image`. Save. Print `wb.sheetnames`.

EXPECTED OUTPUT

```
department
Forge         29148.0
Harbour       30844.0
Library       30910.0
Guard         32264.0
Treasury      32547.0
Apothecary    33017.0
Kitchens      33129.0
```

```
Clockworks    40415.0
Name: annual_salary, dtype: float64
Sheets: ['Data', 'Chart']
Saved my_work/output/report_with_chart.xlsx
```

Bug hunt Pages from the Grimoire, page 11

15 min · `bughunt/report_bug.py`

Dalvric's last working before he sailed west: "the Privy Council's pack in one click". The Queen clicked. Nothing happened. There are **three** bugs: one crashes, one shows the wrong people with a straight face, and one gives a number that's a bit too low (think about zeros).

EXPECTED OUTPUT ONCE FIXED

```
Top 3 earners:
TM1129 Linnet Quickwater 68750
TM1144 Ivo Nightingale 63650
TM1112 Silas Pennyworth 61750
Average bonus of those who got one: 1,934.22 cr
Saved my_work/output/council_pack.xlsx
```

One commit per bug, with a message that explains it.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.